Lecture 12: Convolutional neural networks and PCA

Machine Learning 2026

Federico Chiariotti (federico.chiariotti@unipd.it)

Università degli Studi di Padova

Recap



UNIVERSITÀ
DEGLI STUDI
DI PADOVA

Convolutional

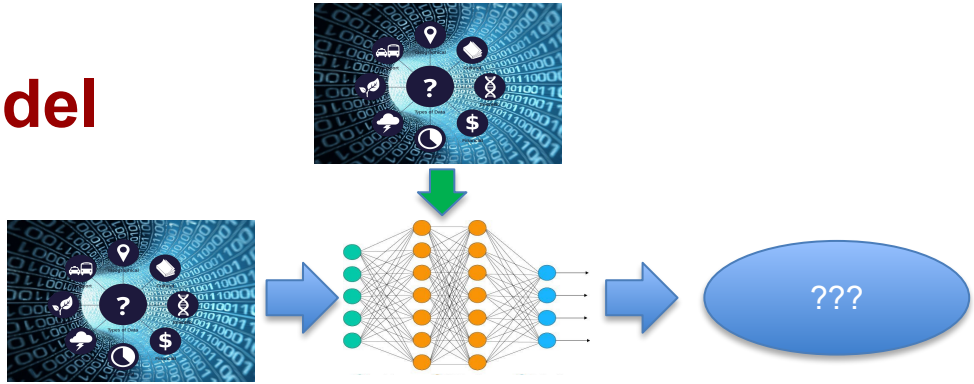
First layer: the receptive field is 3x3, and we only have 2 feature maps

The number of parameters in convolutional layers does not depend on the input size, and is just 2x3x3=18, as these parameters are shared over the whole image

Second layer: each feature map has the same size as the input image, as we used stride 1 and padding. The max pooling layer then reduces the image from 6x6 to 3x3

Third layer: we flatten the two feature maps to 3x3x2=18 parameters. As there are 4 classes, the fully connected layer has 18x4=72 parameters (if we add a bias to the flattened vector, we would have 19x4=76). The CNN has 90 parameters.

Unsupervised learning model

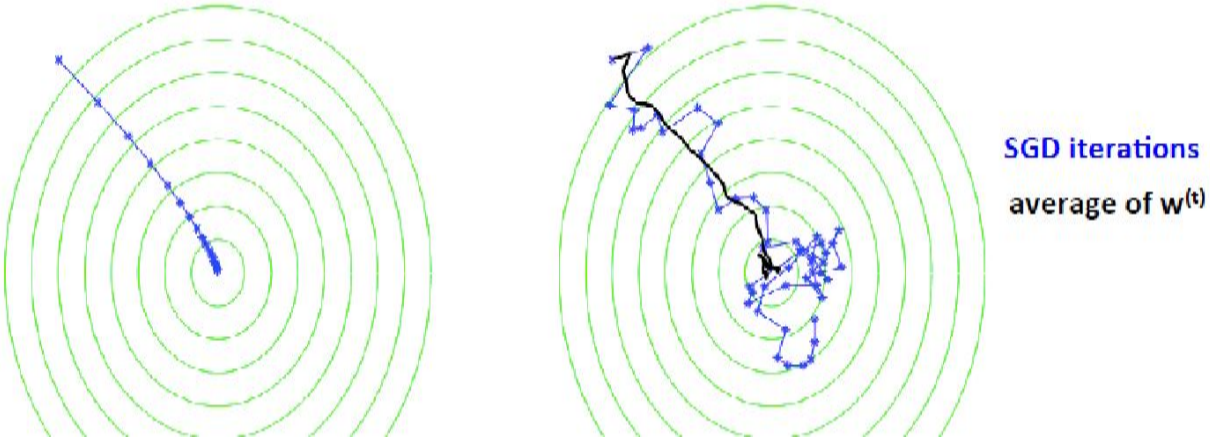


- Domain set X (there is no label set!)
  - We talk about *clustering* if the set is discrete, compression if it is continuous
  - Generative approaches also use unsupervised models
- Data generation model
  - D is a distribution over X **(unknown to the machine learning algorithm)**
  - Training set: sampling according to D (there are no labels!)
- Success metric
  - We do not have an immediate loss function, so we have to find a plausible one!

Stochastic Gradient Descent

GD is computationally expensive: we need to compute the gradient over all points!

We can use a random point (selected from a distribution whose expected value is equal to the gradient) to converge to the same result with a lower cost



Stochastic Gradient Descent

SGD algorithm:

$\boldsymbol{w}^{(0)} \leftarrow \mathbf{0}$

for $t \leftarrow 0$ to $T-1$ do

choose $\boldsymbol{v_t}$ at random from a distribution such that $\mathbb{E}\left[\boldsymbol{v_t}\middle|\boldsymbol{w}^{(t)}\right] = \nabla f(\boldsymbol{w}^{(t)})$

$\boldsymbol{w}^{(t+1)} = \boldsymbol{w}^{(t)} - \eta \boldsymbol{v_t}$

return $\bar{\boldsymbol{w}} = \boldsymbol{w}^{(T)}$ (or $\bar{\boldsymbol{w}} = \frac{1}{T-T_0}\sum_{t=T_0}^{T} \boldsymbol{w}^{(t)}$)

The cost of each iteration is reduced by a factor m

We can apply the same principle in the case of regularized loss and other risk minimization frameworks (e.g., SRM)

Adaptive schemes for SGD

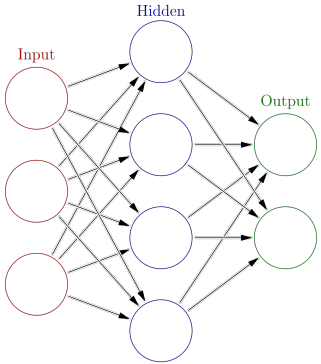
- ➔ **Adagrad** adapts the learning rate for each parameter independently, with higher rates for parameters associated with features that appear rarely
- ➔ **Adam** (adaptive moment estimation) combines Adagrad and momentum, behaving like a rolling ball with friction: it prefers flatter minima, with some regularization

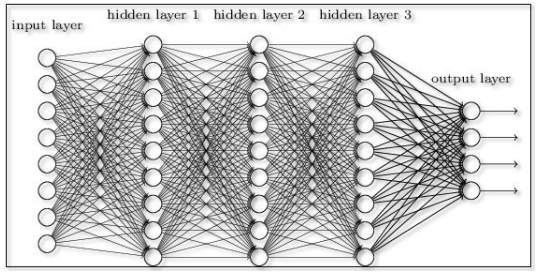
Adam is the most common method in neural network training, but there are multiple variants. The optimizer can be seen as another component of the learning algorithm, and thus selected with cross-validation!

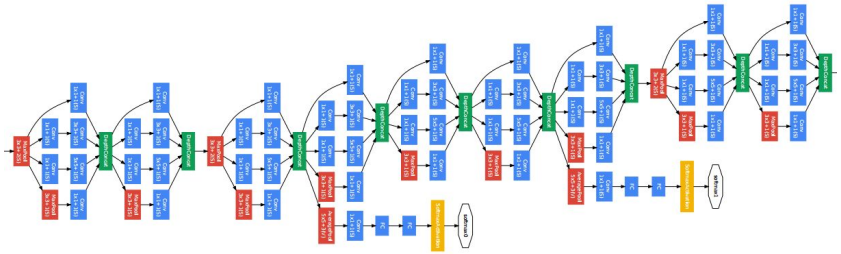


Artificial neural networks

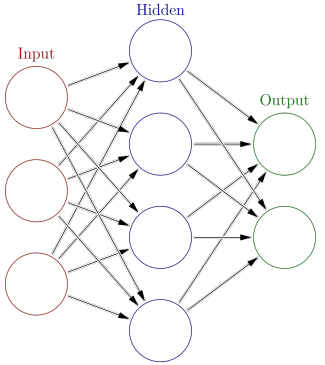
Neural networks are inspired by biological neurons, like the perceptron. Although the idea has been around since the 1980s, practical results surpassed SVMs only in the 2000s. The past 15-20 years have seen a real deep neural network boom, with impressive performance gains







Neural network model



Weighted directed graph $G = (V, E, \mathbf{w})$

- ➔ V is the set of neurons
- ➔ $E \subseteq V \times V$ is the set of connections
- ➔ $\mathbf{w} : E \to \mathbb{R}$ is the weight function (the weights are the parameters to be learned!)

For each neuron v:

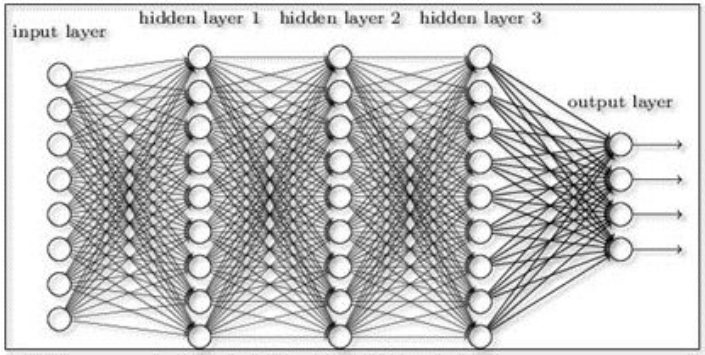
1. The input i_v is the sum of the outputs of the incoming connections, weighted by their edge weight: $i_v = \sum_{u \in V:(u,v) \in E} w_{(u,v)} o_u$
2. The output o_v is the result of an activation function $\sigma(i_v)$

Feedforward neural networks

We can impose a structure to simplify training: the network is organized in **layers**, and each layer only takes the outputs of the previous layer as inputs

We then partition the neurons: $V = \bigcup_{t=0}^{T} V_t, \quad V_t \cap V_{t'} = \varnothing \; \forall t \neq t'$

We have $(u, v) \in E \implies \exists t : u \in V_t \wedge v \in V_{t+1}$: each neuron is connected only to neurons at the next layer



Neurons

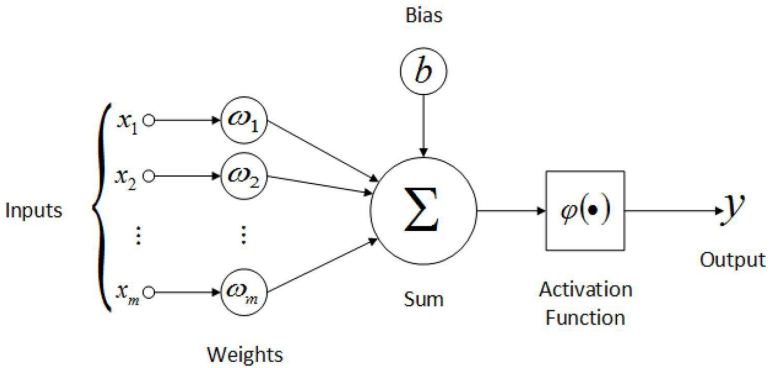
The output $o_{t,i}(\mathrm{x})$ of the i-th neuron in the t-th layer is computed recursively. We also use $\mathbf{v}^{(t)}$ to indicate the output of the whole layer in a compact form.

The input to the neuron is $a_{t+1,j} = \langle \mathbf{w}_j^{(t+1)}, \mathbf{v}^{(t)} \rangle$

The output of the neuron is $o_{t+1,j} = \sigma(a_{t+1,j})$

$$\mathbf{w}^{(t)} = \begin{pmatrix} w_{01}^{(t)} & w_{02}^{(t)} & \cdots & w_{0|V_t|}^{(t)} \\ w_{11}^{(t)} & w_{12}^{(t)} & \cdots & w_{1|V_t|}^{(t)} \\ \vdots & \vdots & \vdots & \vdots \\ w_{|V_{t-1}|1}^{(t)} & w_{|V_{t-1}|2}^{(t)} & \cdots & w_{|V_{t-1}||V_t|}^{(t)} \end{pmatrix}$$

$\mathbf{w}_2^{(t)}$



Neurons

Computing the input of the first layer is simple:

$$a_{1,j} = w_{0,j}^{(1)} + \sum_{i=1}^{\ell(\mathbf{x})} w_{i,j}^{(1)} x_i$$

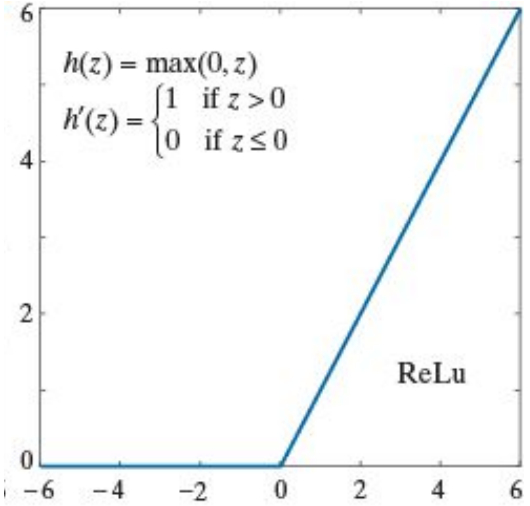
The following layers work recursively:

$$a_{t,j} = w_{0,j}^{(t)} + \sum_{i=1}^{|V_{t-1}|-1} w_{i,j}^{(t)} o_{t-1,i}$$

Activation functions: rectilinear unit

ReLU: most commonly used activation function

➔ (Almost) smooth output
➔ Computationally simple
➔ Gradients do not vanish



Putting it all together: forward propagation

Input: $\mathbf{x} = (x_1, \ldots, x_d)^T$; NN with 1 output node
Output: prediction y of NN;

$v^{(0)} \leftarrow (1, x_1, \ldots, x_d)^T$;
for $t \leftarrow 1$ **to** T **do**
    $a^{(t)} \leftarrow \left(w^{(t)}\right)^T v^{(t-1)}$;
    $v^{(t)} \leftarrow \left(1, \sigma\left(a^{(t)}\right)^T\right)^T$;
$y \leftarrow v^{(T)}$;
return y;

A practical learning algorithm

Target of ERM: minimize the empirical risk

$$\mathbf{w}^* = \arg\min_{\mathbf{w}} \frac{1}{m} \sum_{i=1}^{m} \ell(y_i, h_{\mathbf{w}}(\mathbf{x}_i))$$

We can use SGD to follow the gradients and approximate a solution, as the problem cannot be solved in polynomial time

1. Compute the predicted output
2. **Compute the gradient**
3. Perform SGD

The gradient is with respect to all weights, but the output is after the last layer!

Backpropagation: putting it all together

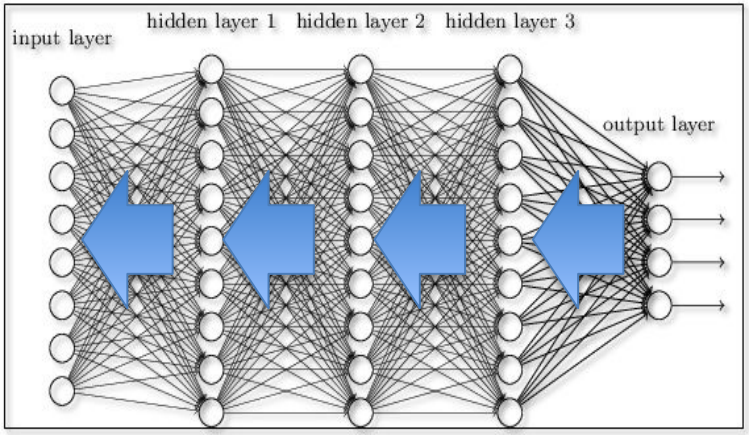
$$\delta_j^{(t)} = \frac{\partial L_S}{\partial o_j^{(t)}} \frac{\partial o_j^{(t)}}{\partial a_j^{(t)}} = \frac{\partial L_S}{\partial o_j^{(t)}} \sigma'(a_j^{(t)})$$

$$\frac{\partial L_S}{\partial o_j^{(t)}} = \sum_{k=1}^{|V_{t+1}|-1} w_{jk}^{(t+1)} \delta_k^{(t+1)}$$

We can compute gradients recursively:

➔ Base step $\delta^{(T)} = \frac{1}{B} \sum_{i=1}^{B} \frac{\partial \ell(y_i, \hat{y}(\mathbf{x}_i))}{\partial \hat{y}(\mathbf{x}_i))}$

➔ Recursive update $\delta_j^{(t)} = \sigma'(a_j^{(t)}) \sum_{k=1}^{|V_{t+1}|-1} w_{jk}^{(t+1)} \delta_k^{(t+1)}$



Backpropagation pseudocode

Input: data point $(\mathbf{x}_i, y_i)$, NN (with weights $w_{ij}^{(t)}$, for $1 \leq t \leq T$)

Output: $\delta^{(t)}$ for $t = 1, \ldots, T$

compute $a^{(t)}$ and $v^{(t)}$ for $t = 1, \ldots T$;

$\delta^{(L)} \leftarrow \frac{\partial L}{\partial a^{(L)}}$;

for $t = T - 1$ **downto** 1 **do**

  $\delta_j^{(\ell)} \leftarrow \sigma'(a_{t,j}) \cdot \sum_{k=1}^{d^{(\ell+1)}} w_{jk}^{(t+1)} \delta_k^{(t+1)}$ for all $j = 1, \ldots, d^{(t)}$;

return $\delta^{(1)}, \ldots, \delta^{(T)}$;

SGD with backpropagation

BackPropagation algorithm with SGD

Input: training data $(x_1, y_1), \ldots, (x_m, y_m)$

Output: NN weights $w_{ij}^{(t)}$

Initialize $w_{ij}^{(t)} \quad \forall i, j, t$;

for $s \leftarrow 0,1,2, \ldots$ **do** // until convergence

pick (x_k, y_k) at random from training data; // SGD: 1 sample at random

compute $v_{t,j} \quad \forall j, t$; // forward propagation

compute $\delta_j^{(t)} \quad \forall j, t$; // backward propagation

$w_{ij}^{(t)[s+1]} = w_{ij}^{(t)[s]} - \eta v_{t-1,i} \delta_j^{(t)} \quad \forall i, j, t$; // update weights

if *converged* **then return** $w_{ij}^{(t)[s]} \quad \forall i, j, t$;

Part 1:
Convolutional neural networks



UNIVERSITÀ
DEGLI STUDI
DI PADOVA

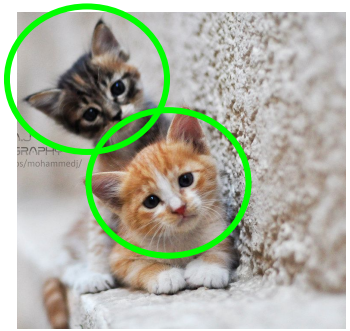
The limits of fully connected models

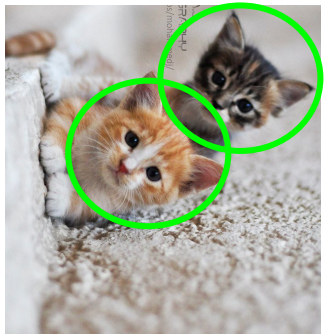
Fully connected NNs have too many weights for large inputs (e.g., images): the number of parameters grows **quadratically**

We can exploit **structure** in some domains (audio, text…)

➔ Structures in images are often local and shift- and rotation-invariant: can we use this?

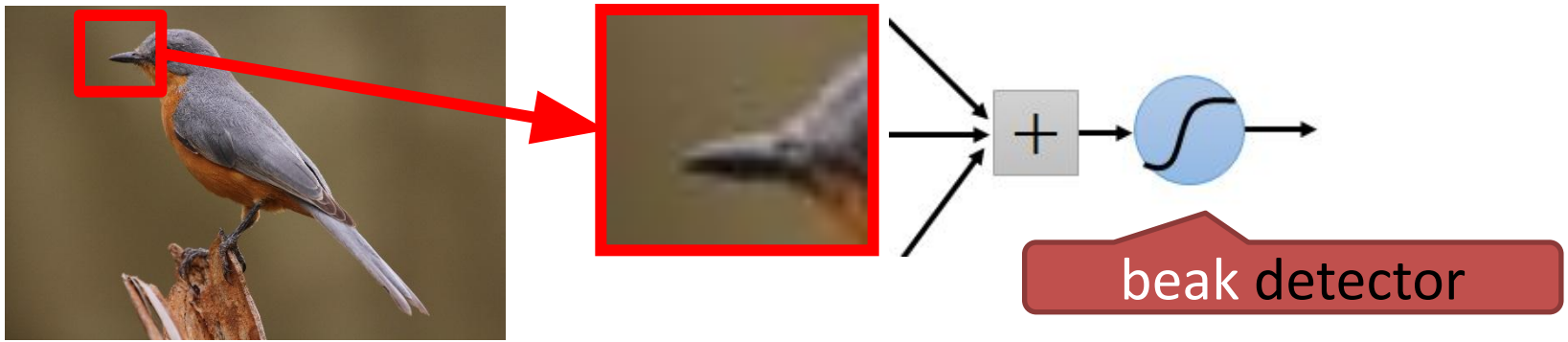






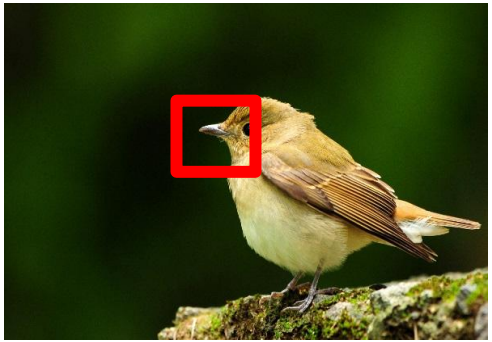
Learning over images

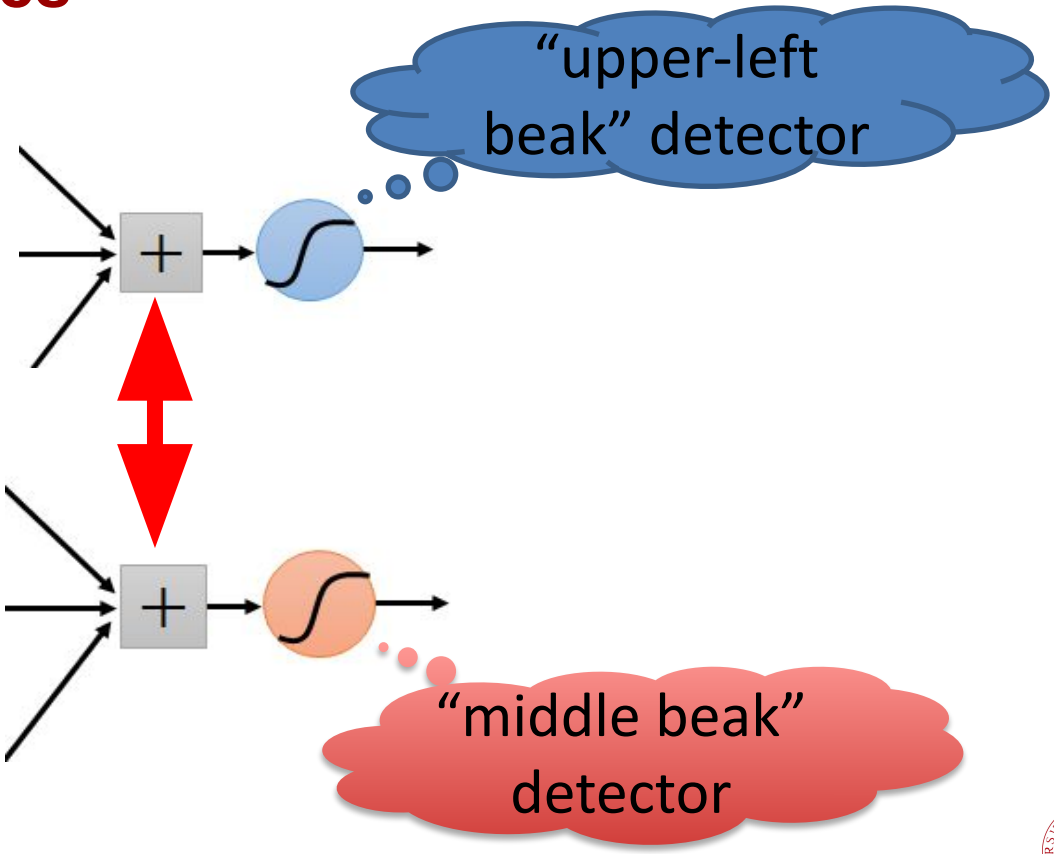
Some patterns are usually smaller than the whole picture, and can help us recognize objects



Learning over images





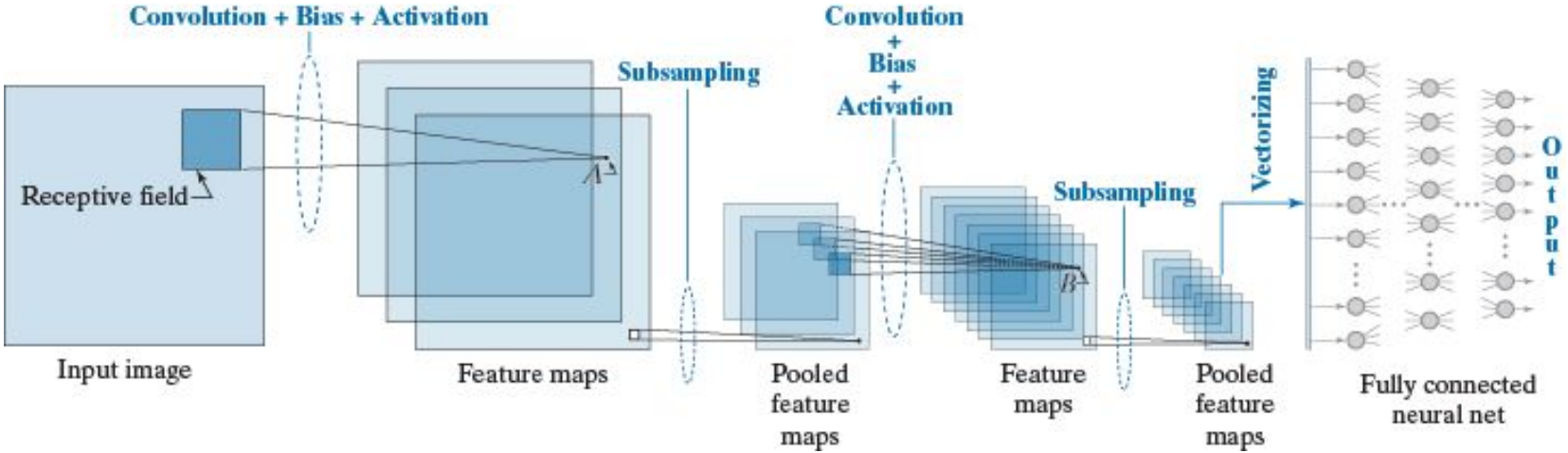


Learning over images

The pattern does not change its meaning because it’s in a different place in the image:

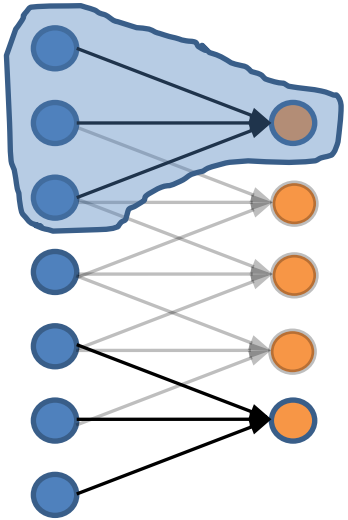
➔ Detectors for the same pattern can share the same parameters
➔ We can use a single detector and then move it around

Convolutional Neural Networks (CNNs)

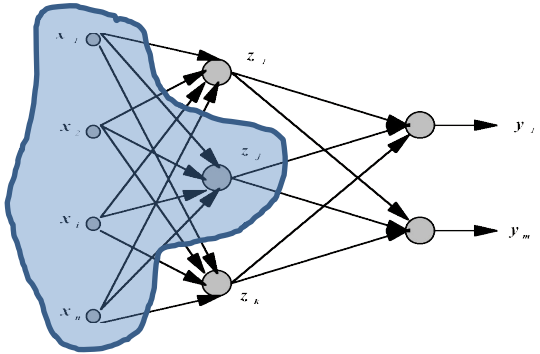


1. Local connectivity: we define a receptive field for each neuron
2. Shared weights: the response is spatially invariant (a beak is a beak wherever it is)
3. Feature maps
4. Subsampling (pooling) to consider different scales

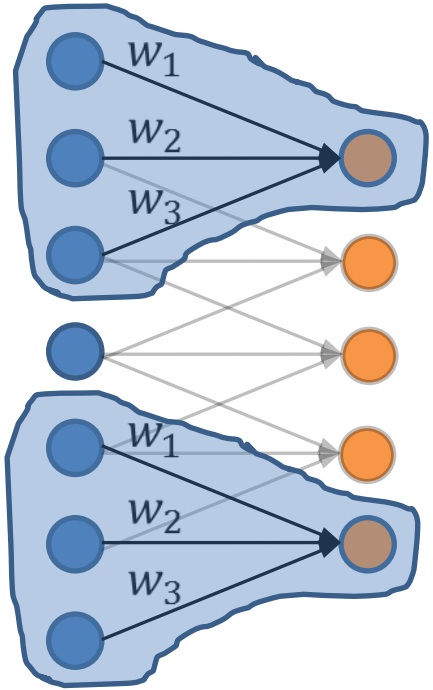
Local connectivity



Each orange unit is only connected to 3 blue units, unlike in a fully connected model

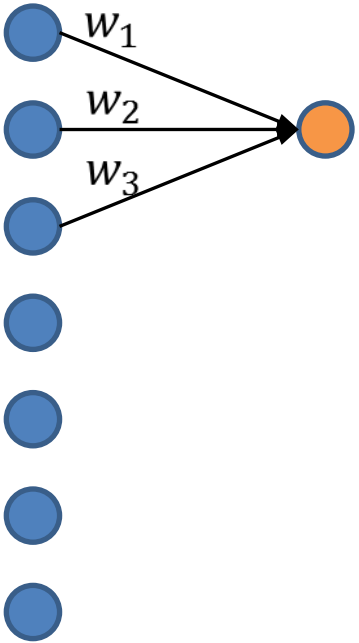


Shared weights



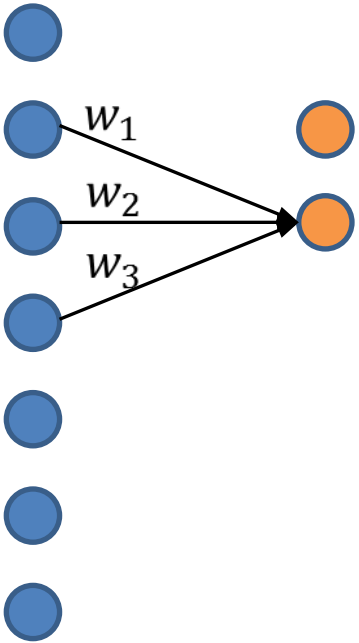
All orange units share the same weights: they compute the same function over a different input

Shared weights



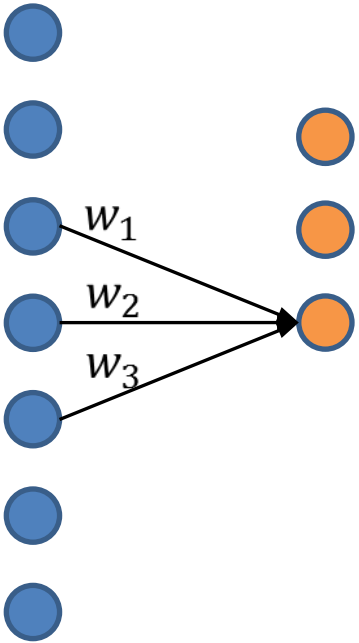
All orange units share the same weights: they compute the same function over a different input

Shared weights



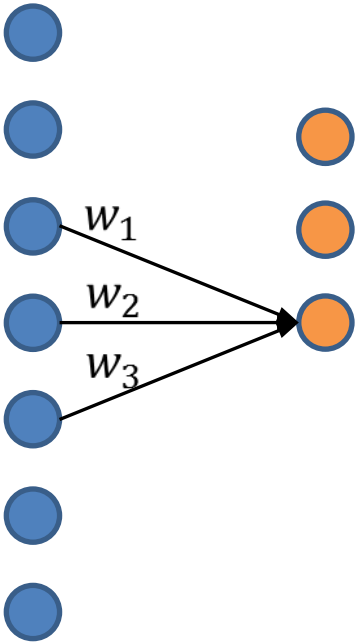
All orange units share the same weights: they compute the same function over a different input

Shared weights



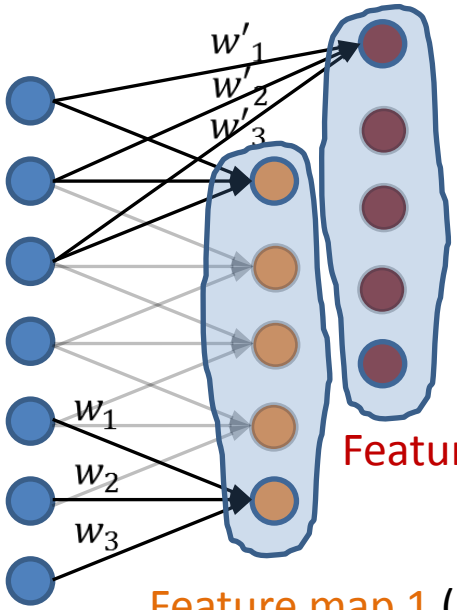
All orange units share the same weights: they compute the same function over a different input

Shared weights



All orange units share the same weights: they compute the same function over a different input

Feature maps

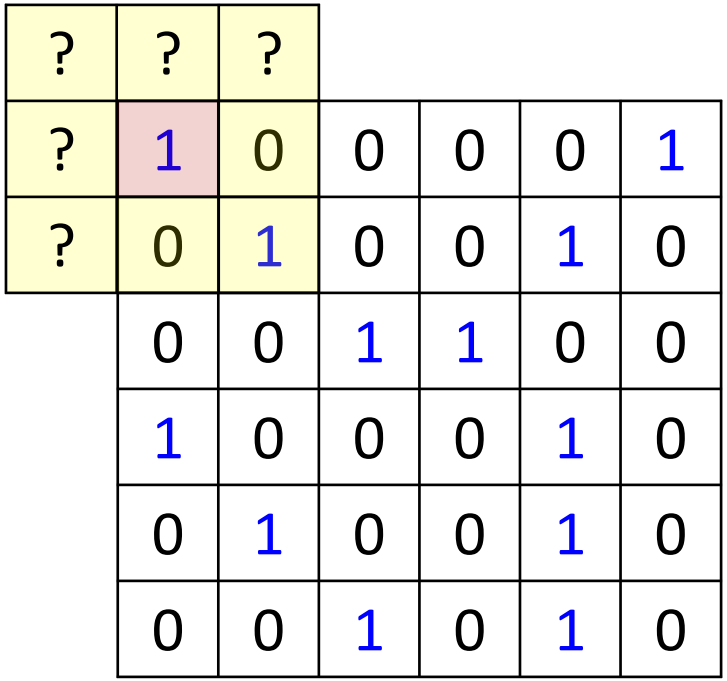


Orange and red units
compute different functions

Feature map 2 (array of red units)

Feature map 1 (array of orange units)

Convolution



We have a 3x3 input, and we shift it over the image.

1. Stride: how much do we shift by?
2. Boundaries: do we stop before (reducing the output size) or pad with zeros (extending the input?)

Convolution

?	?	?				
?	1	0	0	0	0	1
?	0	1	0	0	1	0
	0	0	1	1	0	0
	1	0	0	0	1	0
	0	1	0	0	1	0
	0	0	1	0	1	0

These are the network parameters to be learned.

1	-1	-1
-1	1	-1
-1	-1	1

Filter 1
(feature map 1)

-1	1	-1
-1	1	-1
-1	1	-1

Filter 2
(feature map 2)

Convolution: example

1	-1	-1
-1	1	-1
-1	-1	1

Filter 1

1	0	0	0	0	1
0	1	0	0	1	0
0	0	1	1	0	0
1	0	0	0	1	0
0	1	0	0	1	0
0	0	1	0	1	0

Dot product → 3

Convolution: example

1	-1	-1
-1	1	-1
-1	-1	1

Filter 1

1	0	0	0	0	1
0	1	0	0	1	0
0	0	1	1	0	0
1	0	0	0	1	0
0	1	0	0	1	0
0	0	1	0	1	0

Dot product



stride=1

Convolution: example

1	-1	-1
-1	1	-1
-1	-1	1

Filter 1

1	0	0	0	0	1
0	1	0	0	1	0
0	0	1	1	0	0
1	0	0	0	1	0
0	1	0	0	1	0
0	0	1	0	1	0

Dot product



stride=2

Convolution: example

1	-1	-1
-1	1	-1
-1	-1	1

Filter 1

1	0	0	0	0	1
0	1	0	0	1	0
0	0	1	1	0	0
1	0	0	0	1	0
0	1	0	0	1	0
0	0	1	0	1	0

Dot product →

3	-1	-3	-1
-3	1	0	-3
-3	-3	0	1
3	-2	-2	-1

Convolution: example

1	-1	-1
-1	1	-1
-1	-1	1

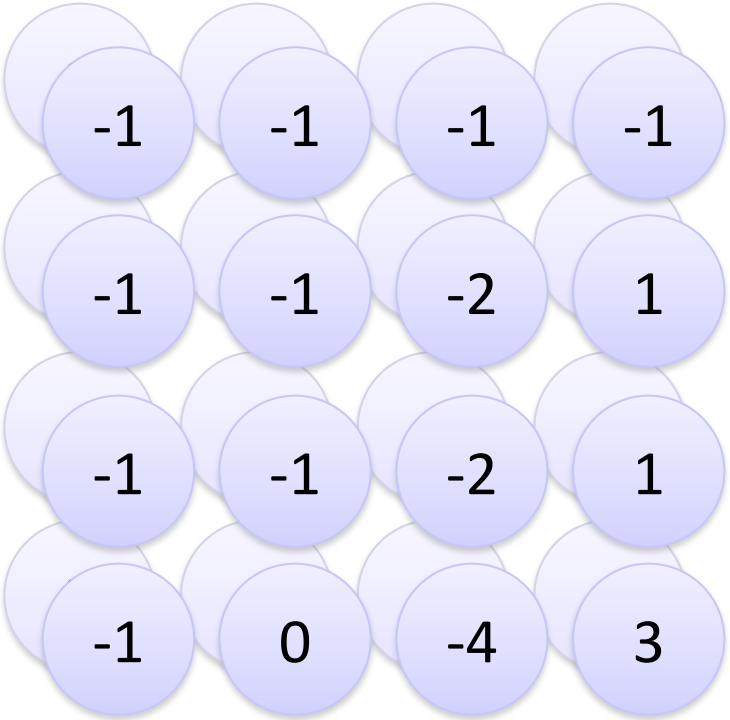
Filter 2

1	0	0	0	0	1
0	1	0	0	1	0
0	0	1	1	0	0
1	0	0	0	1	0
0	1	0	0	1	0
0	0	1	0	1	0

Dot product →

-1	-1	-1	-1
-1	-1	-2	1
-1	-1	-2	1
-1	0	-4	3

Convolution outputs

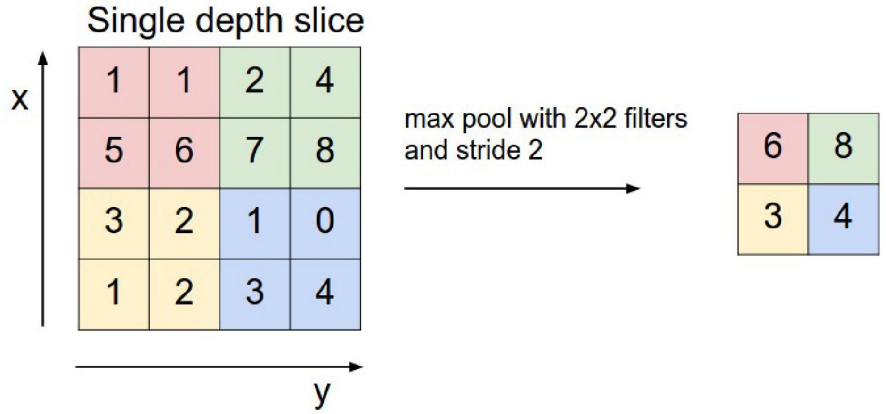


If we have F filters, we get:

- ➔ An Fx4x4 output (with stride 1 and without padding)
- ➔ An Fx6x6 output (with stride 1 and with padding)
- ➔ An Fx4x4 output (with stride 2 and with padding)

There are only 3x3xF learnable parameters!

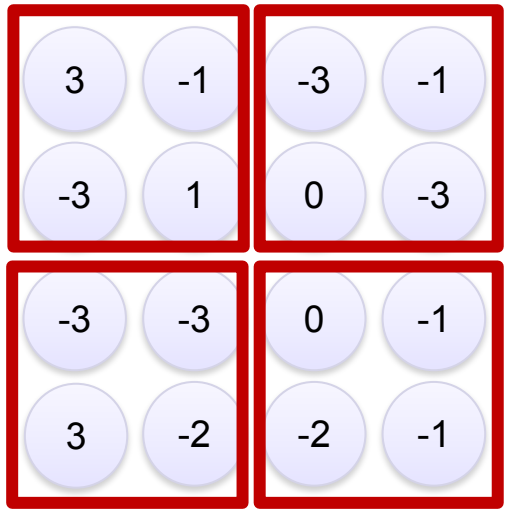
Pooling layers

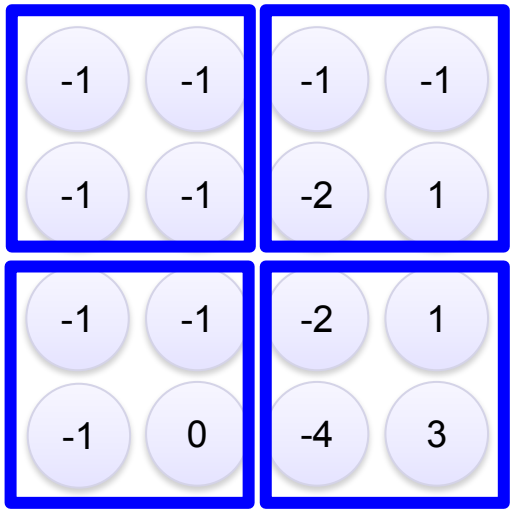


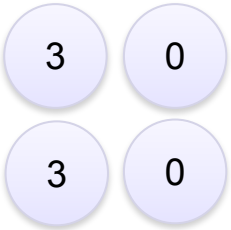
We reduce the resolution, looking for larger-scale features: it also reduces computational costs and adds some deformation invariance

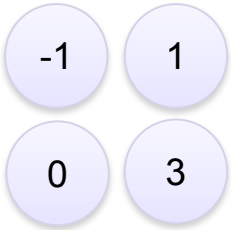
Max pool: take the maximum value in each receptive field

Pooling example





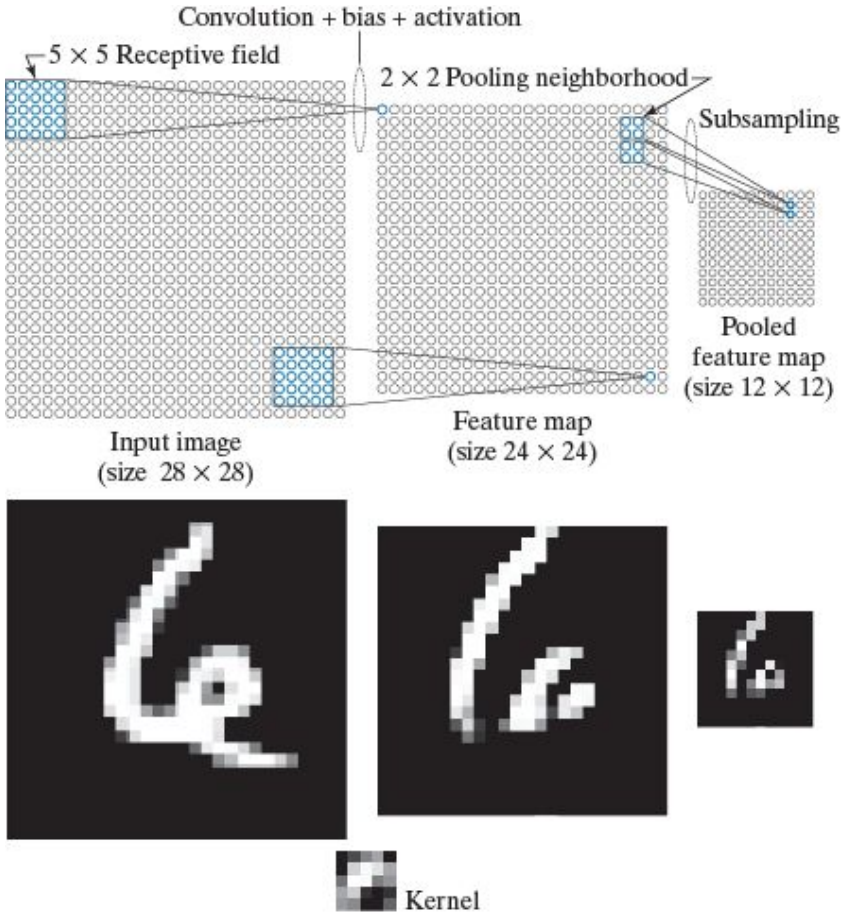




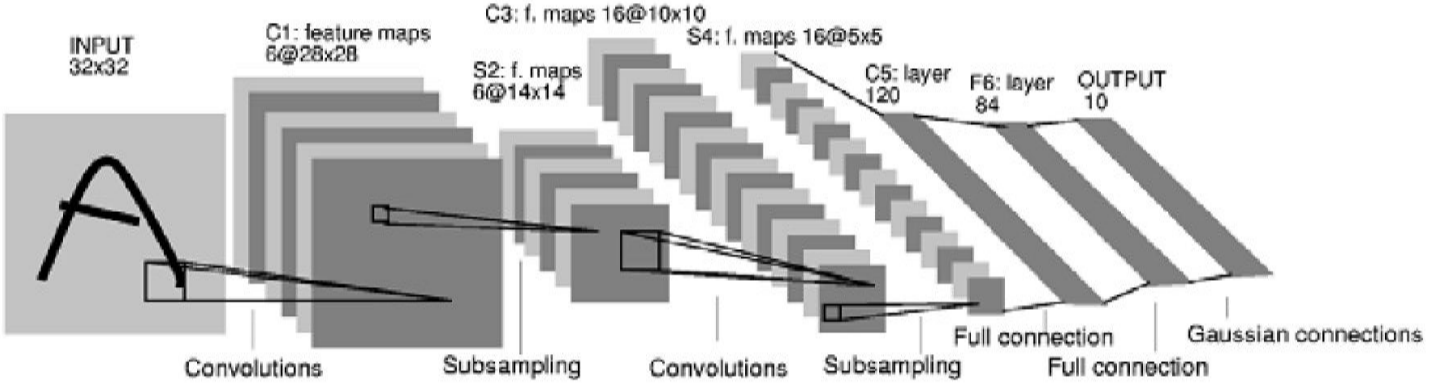
Pooling benefits







Deep CNNs



- ➔ Hierarchical representation: early layers have small, low-level features, later layers have more abstract and larger features
- ➔ We reduce the number of parameters by sharing weights, reducing connections, and pooling

Example: counting weights

Let us consider a 16x16 grayscale image. We have 4 classes, and a fully connected NN with 2 hidden layers with 11 and 9 neurons (including bias neurons).

We can count the weights easily:

First layer: (256+1)x10=2570
Second layer: 11x8=88
Output layer: 9x4=36

Total: 2694 parameters!

Example: counting weights

For the same task, we can now use a CNN. The receptive field is 3x3, with stride 1 and padding. We use 10 filters in the first layer, then do max pooling with a 2x2 field and stride 2 without padding. The second layer has 8 filters, with a 3x3 receptive field, stride 1 and no padding. We then vectorize the output and have a fully connected layer with 9 neurons (including bias).

First CNN layer:

➔ 3x3x10=90 learnable parameters
➔ Output: 10x16x16
➔ Max pooling output: 10x8x8 (stride 2 with no padding)

Example: counting weights

For the same task, we can now use a CNN. The receptive field is 3x3, with stride 1 and padding. We use 10 filters in the first layer, then do max pooling with a 2x2 field and stride 2 without padding. The second layer has 8 filters, with a 3x3 receptive field, stride 1 and padding. We then add another max pooling layer. Finally, we vectorize the output and have a fully connected layer with 9 neurons (including bias).

Second convolutional layer:

- ➔ 3x3x8=72 learnable parameters
- ➔ Output: 8x8x8
- ➔ Max pooling output: 8x4x4 (stride 2 with no padding)



Example: counting weights

For the same task, we can now use a CNN. The receptive field is 3x3, with stride 1 and padding. We use 10 filters in the first layer, then do max pooling with a 2x2 field and stride 2 without padding. The second layer has 8 filters, with a 3x3 receptive field, stride 1 and padding. We then add another max pooling layer. Finally, we vectorize the output and have a fully connected layer with 9 neurons (including bias).

Fully connected layers:

- ➔ Input: 8x4x4+1=65 values including bias
- ➔ Fully connected layer: 65x8=520 parameters
- ➔ Output layer: 9x4=36 parameters

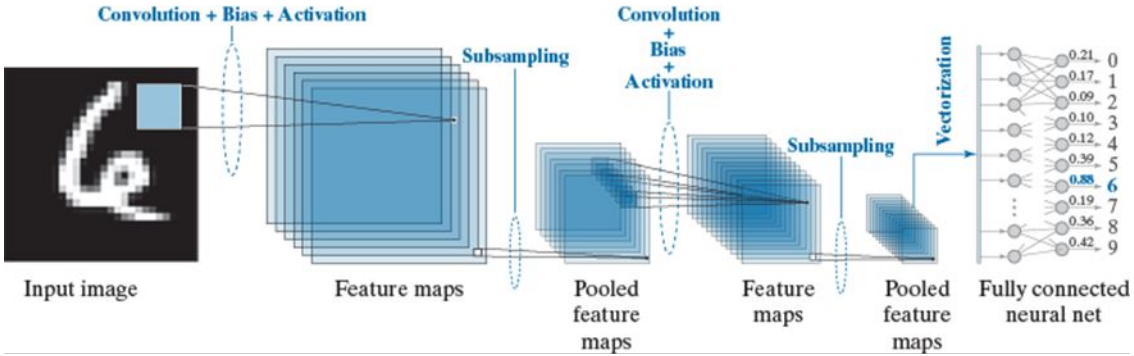
Total parameters

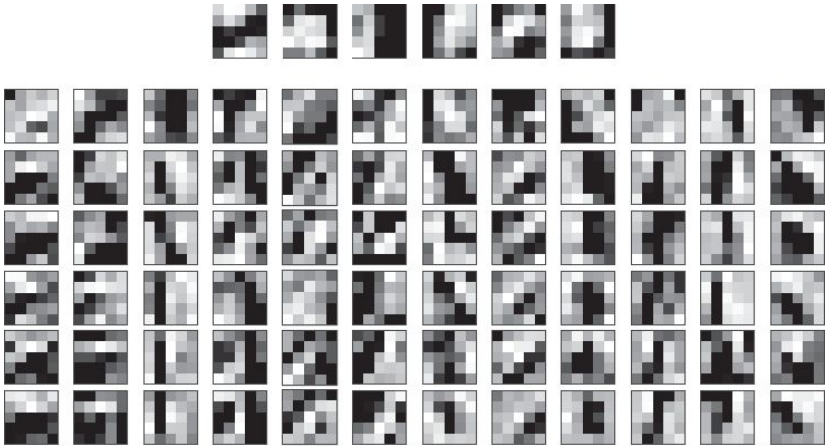
NN layer	Parameters	CNN layer	Parameters
First hidden	2570	First conv.	90
Second hidden	88	First pooling	0
Output	36	Second conv.	72
		Second pooling	0
		Fully connected	520
		Output	36
Total	**2694**	**Total**	**718**

The difference gets bigger with larger images: the convolutional layer scales with the receptive field, not the overall input size



Example: a simple CNN



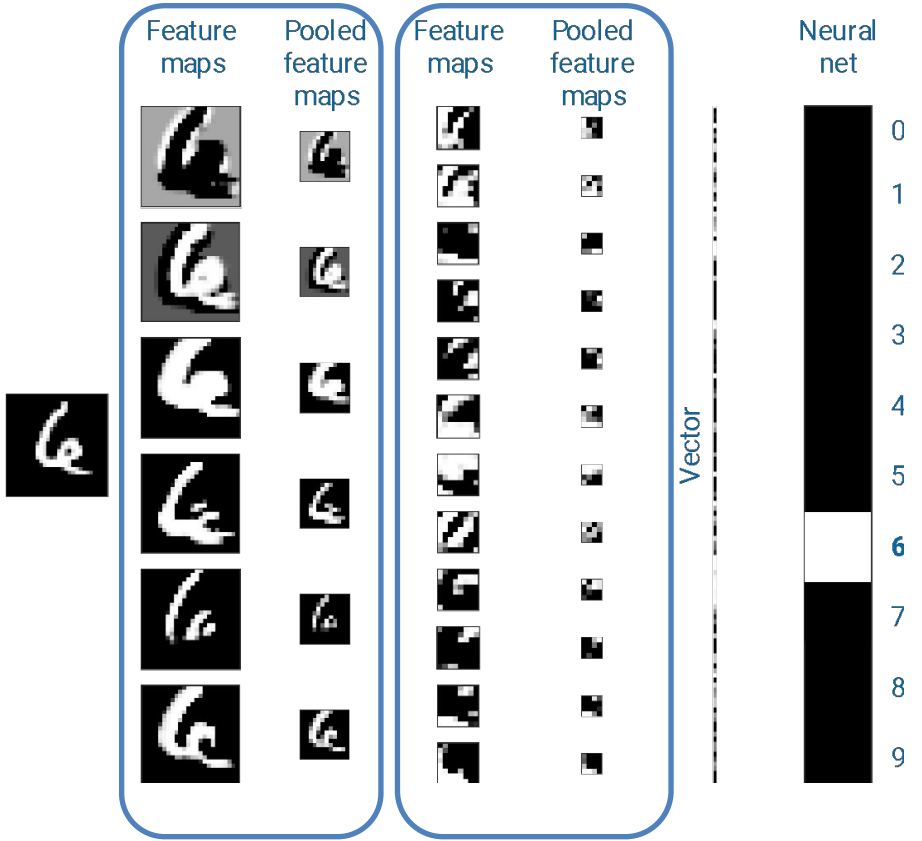


First layer: 5x5 conv., 6 maps
2x2 pooling
Second layer: 5x5 conv., 12 maps
2x2 pooling

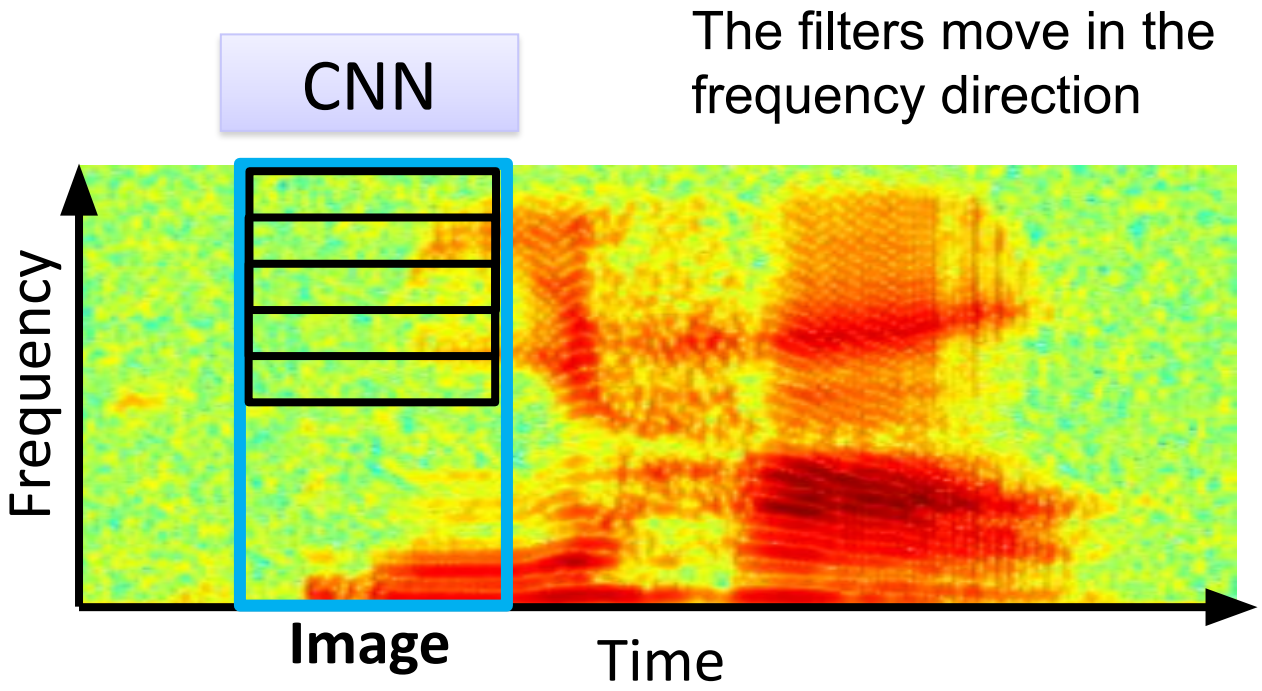
Example: a simple CNN

First layer: simple features, maps still look like the original image

Second layer: abstract features, enough to recognize the digit



CNNs in speech recognition

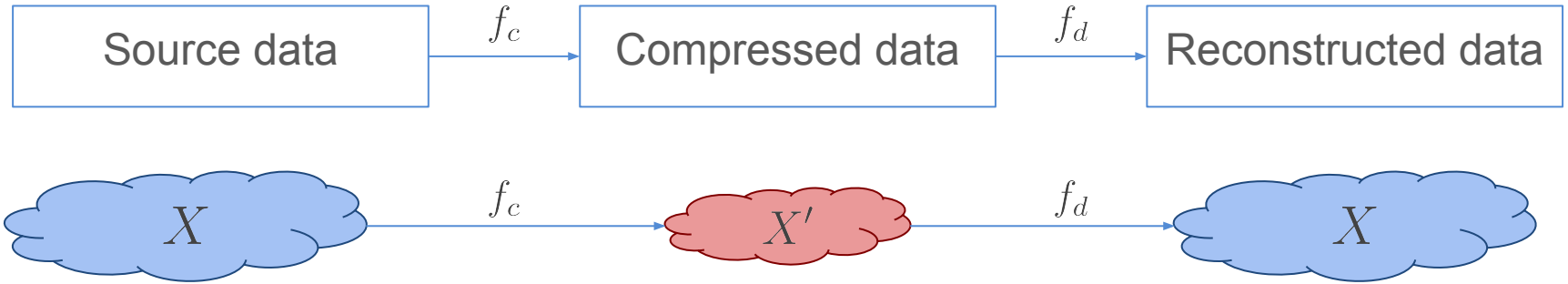


Part 2:
Principal component analysis



UNIVERSITÀ
DEGLI STUDI
DI PADOVA

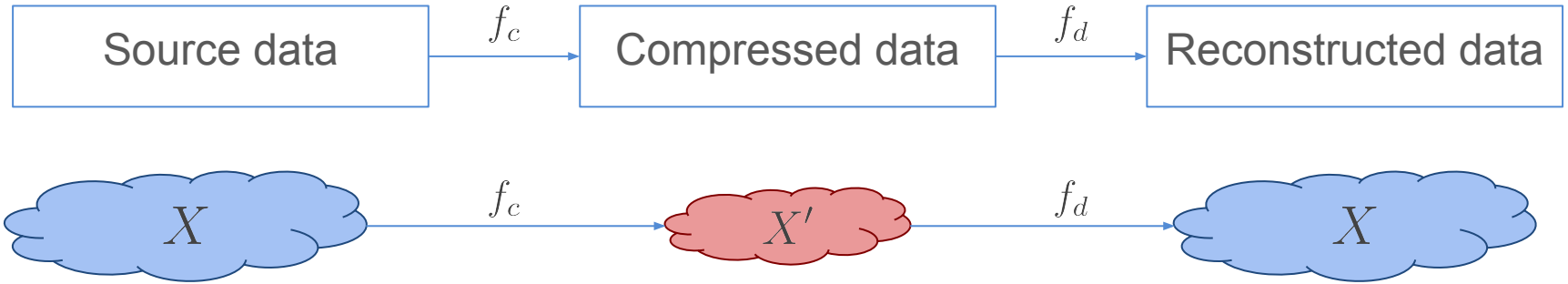
How do we compress data?



We want to go from X to X' (which, ideally, is a smaller space!) through a compression function $f_c : X \to X'$. We also have to define a decompression function $f_d : X' \to X$.

Ideal compression is lossless, i.e., $f_d(f_c(\mathbf{x})) = \mathbf{x},\ \forall \mathbf{x} \in X$

Why do we need this?



- ➔ Feature reduction: we simplify our learning problem by reducing the input space!
- ➔ Capturing correlations to analyze manually
- ➔ Memory! If we compress data, we can save space

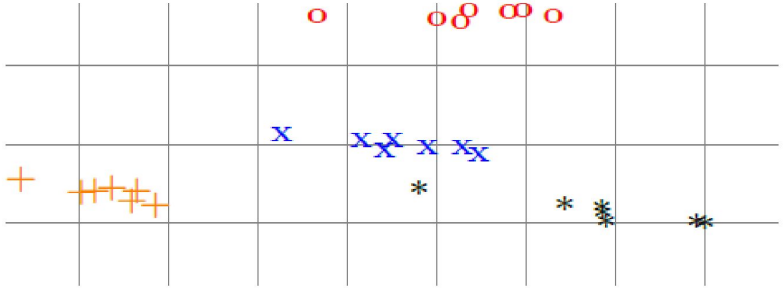
We cannot guarantee lossless compression in practice, but it gives us a hint…

Projection



$$\mathbb{R}^{50x50} = \mathbb{R}^{2500}$$

PCA



We can use algebraic projection to reduce dimensionality while maintaining the most important features: we use MSE as a loss function

Algebraic projection

We have data points $\mathbf{x}_1, \ldots, \mathbf{x}_m \in \mathbb{R}^d$

We define the projection matrix $\mathbf{W} \in \mathbb{R}^{n \times d}$, so that $\mathbf{y} = \mathbf{W}\mathbf{x}$ is a projection of the original point in a smaller space (n<d)

We then also define the inverse mapping $\mathbf{U} \in \mathbb{R}^{d \times n}$, so we can apply both to go back to the original space:

$$\tilde{\mathbf{x}} = \mathbf{U}\mathbf{y} = \mathbf{U}\mathbf{W}\mathbf{x}$$

Principal Component Analysis (PCA)

We have data points $\mathbf{x}_1, \ldots, \mathbf{x}_m \in \mathbb{R}^d$

The reconstructed input is $\tilde{\mathbf{x}} = \mathbf{U}\mathbf{y} = \mathbf{U}\mathbf{W}\mathbf{x}$

PCA optimization problem: find the projection system that minimizes the reconstruction MSE

$$\arg\min_{\mathbf{W},\mathbf{U}} \frac{1}{m} \sum_{i=1}^{m} ||\mathbf{x}_i - \tilde{\mathbf{x}}_i||^2$$

Lemma: optimal inverse

There is an optimal solution $(\mathbf{W}^*, \mathbf{U}^*)$ of the PCA problem such that:

➔ The columns of $\mathbf{U}^*$ are orthonormal, i.e., $(\mathbf{U}^*)^T\mathbf{U}^* = \mathbf{I}$

➔ $(\mathbf{U}^*)^T = \mathbf{W}^*$

This means that we only need to find $\mathbf{U}^*$, and then we can get both matrices almost for free!

Proof: first step

Let us consider the mapping $\mathbf{x} \rightarrow \mathbf{UWx}$

$\mathbf{V} \in \mathbb{R}^{d \times n}$ forms an (incomplete) orthonormal basis of the codomain of the mapping

We also have $\mathbf{V}^T\mathbf{V} = \mathbf{I}$

Proof: second step

$$||\mathbf{x} - \mathbf{V}\mathbf{y}||^2 = ||\mathbf{x}||^2 + \mathbf{y}^T\mathbf{V}^T\mathbf{V}\mathbf{y} - 2\mathbf{y}^T\mathbf{V}^T\mathbf{x}$$

$$\mathbf{V}^T\mathbf{V} = \mathbf{I}$$

We can then substitute this with the squared norm of $\mathbf{y}$:

$$||\mathbf{x} - \mathbf{V}\mathbf{y}||^2 = ||\mathbf{x}||^2 + ||\mathbf{y}||^2 - 2\mathbf{y}^T\mathbf{V}^T\mathbf{x}$$

We can then try to minimize this:

$$\arg\min_{\mathbf{y}} ||\mathbf{x}||^2 + ||\mathbf{y}||^2 - 2\mathbf{y}^T\mathbf{V}^T\mathbf{x}$$

Proof: third step

$$\arg\min_{\mathbf{y}} ||\mathbf{x}||^2 + ||\mathbf{y}||^2 - 2\mathbf{y}^T\mathbf{V}^T\mathbf{x}$$

Let us take the gradient with respect to $\mathbf{y}$ and set it to 0:

$$2\mathbf{y} - 2\mathbf{V}^T\mathbf{x} = 0$$

We then have the following result:

$$\mathbf{y}^* = \mathbf{V}^T\mathbf{x}$$

This means that using the basis is the best possible mapping:

$$\mathbf{V}^T\mathbf{V}\mathbf{x} = \arg\min_{\tilde{\mathbf{x}}} ||\mathbf{x} - \tilde{\mathbf{x}}||^2$$

Proof: final step

We showed that the basis is the best possible mapping:

$$\mathbf{V}^T\mathbf{V}\mathbf{x} = \arg\min_{\tilde{\mathbf{x}}} ||\mathbf{x} - \tilde{\mathbf{x}}||^2$$

We can then substitute it for any pair $\mathbf{U}, \mathbf{W}$ without increasing the error:

$$||\mathbf{U}\mathbf{W}\mathbf{x}||^2 \geq ||\mathbf{V}^T\mathbf{V}\mathbf{x}||^2$$

Since this is true for any input, we can easily see that it also minimizes the MSE over the whole training set:

$$\sum_{i=1}^{m} ||\mathbf{U}\mathbf{W}\mathbf{x}_i||^2 \geq \sum_{i=1}^{m} ||\mathbf{V}^T\mathbf{V}\mathbf{x}_i||^2$$

Theorem: PCA solution

The optimal solution $\mathbf{U}^*$ of the PCA problem is given by the first n eigenvectors (sorted by their eigenvalues in descending order) of matrix $\mathbf{A} = \sum_{i=1}^{m} \mathbf{x}_i \mathbf{x}_i^T$

This makes our life very easy: we have well-known methods (such as singular value decomposition) to find eigenvectors in polynomial time!

Proof: first step

We write the problem as

$$\arg\min_{\mathbf{U}\in\mathbb{R}^{d\times n}:\mathbf{U}^T\mathbf{U}=\mathbf{I}} \sum_{i=1}^{m} ||\mathbf{x}_i - \mathbf{U}\mathbf{U}^T\mathbf{x}_i||^2$$

We can rewrite the objective for a single point:

$$||\mathbf{x} - \mathbf{U}\mathbf{U}^T\mathbf{x}||^2 = ||\mathbf{x}||^2 + \mathbf{x}^T\mathbf{U}\mathbf{U}^T\mathbf{U}\mathbf{U}^T\mathbf{x} - 2\mathbf{x}^T\mathbf{U}\mathbf{U}^T\mathbf{x}$$

$$\mathbf{U}^T\mathbf{U} = \mathbf{I}$$

If we remove the identity, we get the following:

$$||\mathbf{x} - \mathbf{U}\mathbf{U}^T\mathbf{x}||^2 = ||\mathbf{x}||^2 - \mathbf{x}^T\mathbf{U}\mathbf{U}^T\mathbf{x}$$

Proof: second step

$$||\mathbf{x} - \mathbf{U}\mathbf{U}^T\mathbf{x}||^2 = ||\mathbf{x}||^2 - \mathbf{x}^T\mathbf{U}\mathbf{U}^T\mathbf{x}$$

We can get to the following result:

$$||\mathbf{x} - \mathbf{U}\mathbf{U}^T\mathbf{x}||^2 = ||\mathbf{x}||^2 - \mathrm{tr}(\mathbf{U}^T\mathbf{x}\mathbf{x}^T\mathbf{U})$$

The trace is the sum of diagonal entries of a matrix. It is a linear operator, so we can rewrite the problem as

$$\arg\max_{\mathbf{U}} \mathrm{tr}\left(\mathbf{U}^T \sum_{i=1}^{m} \mathbf{x}_i\mathbf{x}_i^T \mathbf{U}\right)$$

$$\mathbf{A} = \sum_{i=1}^{m} \mathbf{x}_i\mathbf{x}_i^T$$

Proof: third step

We have the following problem:

$$\arg\max_{\mathbf{U}} \operatorname{tr}\left(\mathbf{U}^T\mathbf{A}\mathbf{U}\right)$$

Now, let $\mathbf{V}\mathbf{D}\mathbf{V}^T$ be the spectral decomposition of $\mathbf{A}$ (i.e., $\mathbf{V}\mathbf{D}\mathbf{V}^T = \mathbf{A}$).

Diagonal

Orthonormal (rows and columns)

We define $\mathbf{B} = \mathbf{V}^T\mathbf{U}$, and we have $\mathbf{V}\mathbf{B} = \mathbf{V}\mathbf{V}^T\mathbf{U} = \mathbf{U}$

$\mathbf{V}\mathbf{V}^T = \mathbf{I}$

Proof: fourth step

We have $\mathrm{B} = \mathrm{V}^T\mathrm{U}$, $\mathrm{VB} = \mathrm{VV}^T\mathrm{U} = \mathrm{U}$, and $\mathrm{VDV}^T = \mathrm{A}$

$$\mathrm{U}^T\mathrm{AU} = \mathrm{B}^T\mathrm{V}^T\mathrm{VDV}^T\mathrm{VB}$$

Now, we know that $\mathrm{VV}^T = \mathrm{V}^T\mathrm{V} = \mathrm{I}$

We get $\mathrm{U}^T\mathrm{AU} = \mathrm{B}^T\mathrm{DB}$

Proof: fifth step

We have $\mathbf{B} = \mathbf{V}^T\mathbf{U}$, $\mathbf{U}^T\mathbf{A}\mathbf{U} = \mathbf{B}^T\mathbf{D}\mathbf{B}$, and $\mathbf{V}\mathbf{D}\mathbf{V}^T = \mathbf{A}$

We can apply this to the original problem:

$$\mathrm{tr}\left(\mathbf{U}^T\mathbf{A}\mathbf{U}\right) = \sum_{j=1}^{d} D_{j,j} \sum_{i=1}^{n} B_{j,i}^2$$

Now, we look at $\mathbf{B}$:

$$\mathbf{B}^T\mathbf{B} = \mathbf{U}^T\mathbf{V}\mathbf{V}^T\mathbf{U}$$

Since $\mathbf{U}$'s columns and $\mathbf{V}$ are orthonormal, $\mathbf{B}$ also has orthonormal columns!

Proof: sixth step

Since $\mathbf{B}$ has orthonormal columns, we have

$$\sum_{j=1}^{d}\sum_{i=1}^{n} B_{j,i}^2 = n$$

Now, let us construct $\tilde{\mathbf{B}} \in \mathbb{R}^{d\times d}$, whose first n columns are equal to $\mathbf{B}$, and with $\tilde{\mathbf{B}}^T\tilde{\mathbf{B}} = \mathbf{I}$

Since $\sum_{i=1}^{d} \tilde{B}_{j,i} = 1,\ \forall j$, we have $\sum_{i=1}^{d} B_{j,i} \leq 1,\ \forall j$

Proof: final step

We turn the problem into the following:

$$\arg\max_{\beta \in [0,1]^d : ||\beta||_1 \leq n} \sum_{j=1}^{d} D_{j,j} \beta_j$$

It is easy to see that the optimum solution is to set the first n elements to 1, where the eigenvalues are sorted by size, so the result is $\sum_{j=1}^{n} D_{j,j}$

Selecting the first n eigenvalues then gives us our PCA solution!

PCA: pseudocode

PCA

input

A matrix of m examples $X \in \mathbb{R}^{m,d}$

number of components n

if $(m > d)$

$A = X^\top X$

Let $\mathbf{u}_1, \ldots, \mathbf{u}_n$ be the eigenvectors of A with largest eigenvalues

else

$B = XX^\top$

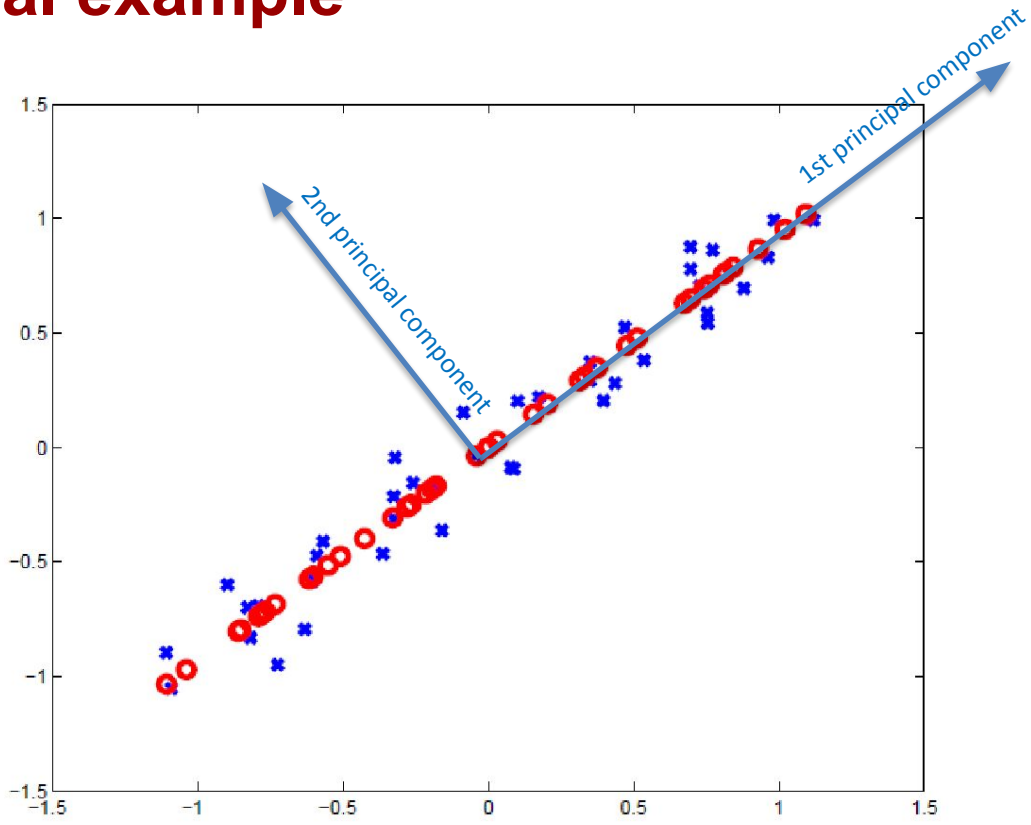
Let $\mathbf{v}_1, \ldots, \mathbf{v}_n$ be the eigenvectors of B with largest eigenvalues

for $i = 1, \ldots, n$ set $\mathbf{u}_i = \frac{1}{\|X^\top \mathbf{v}_i\|} X^\top \mathbf{v}_i$

output: $\mathbf{u}_1, \ldots, \mathbf{u}_n$

Trick for m<d

PCA: graphical example



Part 3: Exercises



Università degli Studi di Padova

Exercise 3.3

Consider the following multi-class classification problem: take in input 6x6 grayscale images (i.e., there are 36 input values for each sample) and classify them into 4 different classes depending on the content using a neural network.

1. A simple 3 layer (input, hidden, output) fully connected feedforward neural network can be used to deal with this problem. Assume that the hidden layer has 10 neurons. What is the number of trainable parameters in the neural network?

2. A Convolutional Neural Network (CNN) that has a single convolutional layer with 3 × 3 convolutions (with stride 1 and padding) and 2 feature maps, followed by 2 × 2 pooling (stride 2, no padding) and then flattens the data and sends it to the output fully connected layer. What is the number of parameters? (Suggestion: be careful of the image boundaries). Would the benefits of this approach be different for images of different size?

Fully connected

First layer: we have 6x6=36 inputs and 10 hidden neurons

We remove the bias neuron, so there are 9 connected neurons. The exercise does not mention an input bias, so we can consider it (in this case, we increase to 37 inputs) or not. If we do not consider it, $w^{(1)}$ has 36x9=324 parameters

Fully connected

First layer: we have 6x6=36 inputs and 10 hidden neurons

We remove the bias neuron, so there are 9 connected neurons. The exercise does not mention an input bias, so we can consider it (in this case, we increase to 37 inputs) or not. If we do not consider it, $w^{(1)}$ has 36x9=324 parameters

Second layer: we have 10 inputs and 4 outputs

In this case, we do not remove bias, as the last layer does not have bias neurons. We just have 10x4=40 parameters in $w^{(2)}$

The total number of parameters is 324+40=364

Convolutional

First layer: the receptive field is 3x3, and we only have 2 feature maps

The number of parameters in convolutional layers does not depend on the input size, and is just 2x3x3=18, as these parameters are shared over the whole image

Convolutional

First layer: the receptive field is 3x3, and we only have 2 feature maps

The number of parameters in convolutional layers does not depend on the input size, and is just 2x3x3=18, as these parameters are shared over the whole image

Second layer: each feature map has the same size as the input image, as we used stride 1 and padding. The max pooling layer then reduces each feature mao from 6x6 to 3x3

Exercise 3.4

Consider the following multi-class classification problem: take as input 12x12 grayscale images and classify them as cats or dogs using a neural network.

1. A fully connected feedforward neural network with 2 hidden layers. The two layers have 25 and 10 neurons, respectively. What is the number of trainable parameters in the neural network?

2. A Convolutional Neural Network (CNN) that has a single convolutional layer with 3 × 3 convolutions (with stride 1 and padding) and 5 feature maps, followed by 2 × (stride 2, no padding). There is another convolutional layer with 3 × 3 convolutions (with stride 1 and padding) and 4 feature maps, followed by another 2 × 2 pooling (stride 2, no padding). The data is then flattened and sent to the output fully connected layer. What is the number of parameters? (Suggestion: be careful of the image boundaries).

Fully connected

First layer: we have 12x12=144 inputs and 25 hidden neurons

We remove the bias neuron, so there are 24 connected neurons. The exercise does not mention an input bias, so we can consider it (in this case, we increase to 145 inputs) or not. If we do not consider it, $w^{(1)}$ has 144x24=3456 parameters

Fully connected

First layer: we have 12x12=144 inputs and 25 hidden neurons

We remove the bias neuron, so there are 24 connected neurons. The exercise does not mention an input bias, so we can consider it (in this case, we increase to 145 inputs) or not. If we do not consider it, $w^{(1)}$ has 144x24=3456 parameters

Second layer: we have 25 inputs and 9 outputs (removing the bias neuron). Matrix $w^{(2)}$ then has 25x9=225 parameters

Fully connected

First layer: we have 12x12=144 inputs and 25 hidden neurons

We remove the bias neuron, so there are 24 connected neurons. The exercise does not mention an input bias, so we can consider it (in this case, we increase to 145 inputs) or not. If we do not consider it, $w^{(1)}$ has 144x24=3456 parameters

Second layer: we have 25 inputs and 9 outputs (removing the bias neuron). Matrix $w^{(2)}$ then has 25x9=225 parameters

Output layer: we have 2 classes, so there are 10x2=20 connections.

The total number of parameters is 3456+225+20=3701 parameters

Convolutional

First layer: the receptive field is 3x3, and we have 5 feature maps

The number of parameters in convolutional layers does not depend on the input size, and is just 5x3x3=45, as these parameters are shared over the whole image

Convolutional

First layer: the receptive field is 3x3, and we have 5 feature maps

The number of parameters in convolutional layers does not depend on the input size, and is just 5x3x3=45, as these parameters are shared over the whole image

Pooling layer: each feature map is the same size as the input, but using stride 2 and no padding reduces it to 6x6

Convolutional

First layer: the receptive field is 3x3, and we have 5 feature maps

The number of parameters in convolutional layers does not depend on the input size, and is just 5x3x3=45, as these parameters are shared over the whole image

Pooling layer: each feature map is the same size as the input, but using stride 2 and no padding reduces it to 6x6

Second layer: the receptive field is 3x3, and we have 4 feature maps

The number of parameters in convolutional layers does not depend on the input size, and is just 4x3x3=36, as these parameters are shared over the whole image

Convolutional

Second layer: the receptive field is 3x3, and we have 4 feature maps

The number of parameters in convolutional layers does not depend on the input size, and is just 4x3x3=36, as these parameters are shared over the whole image

Second pooling layer: each feature map is 6x6, but using stride 2 and no padding reduces it to 3x3

Convolutional

Second layer: the receptive field is 3x3, and we have 4 feature maps

The number of parameters in convolutional layers does not depend on the input size, and is just 4x3x3=36, as these parameters are shared over the whole image

Second pooling layer: each feature map is 6x6, but using stride 2 and no padding reduces it to 3x3

Fully connected layer: we have 4 3x3 maps, so 48 outputs after flattening. There are 2 classes, so we get 48x2=96 parameters (98 if we add a bias).

The CNN has 45+36+96=177 parameters